

SYSTEM DESIGN + INTERVIEW PREPARATION

vkassist RAG Pipeline

A code-grounded explanation of the Vulkan documentation retrieval-augmented generation system: design decisions, trade-offs, alternatives, evaluation, and honest production limitations.

Repository scope: PDF ingestion → structure-aware chunks → pgvector + BM25 hybrid retrieval → RRF → cross-encoder reranking → parent context → cited answer generation.

1. The one-minute explanation

vkassist is a retrieval-augmented generation system for Vulkan documentation. It does not rely on an LLM's memory to answer technical questions. It first extracts and indexes the source PDF, retrieves evidence with both semantic and lexical search, fuses and optionally reranks candidates, expands the winning chunk with local or parent context, and finally asks a model to answer only from those passages with numbered citations.

OFFLINE INGESTION

```
PDF → PyMuPDF layout blocks + TOC → sections (parents) + bounded child chunks
    → embedding text (section/page/API symbols/content) → BGE vectors → PostgreSQL + pgvector
    → raw chunk text → BM25 in-memory index
```

ONLINE QUERY

```
query → BGE query vector —————→
    → pgvector cosine top 20          |
query → identifier-aware BM25 top 20 —→ Reciprocal Rank Fusion → top candidates
                                     ↳ Cross-encoder reranker → top 5
top child → adjacent children / full parent → grounded LLM prompt → answer + cited pages
```

Interview framing: “I separated retrieval quality from generation quality. The LLM is the final writer, not the search engine; evidence retrieval is deterministic enough to benchmark independently.”

2. Corpus and ingestion: why a PDF needs more than text splitting

The source is `data/vulkan_documentation.pdf`. PDFs encode visual layout, not a reliable semantic document tree. Naively calling `page.get_text()` and cutting every N characters creates chunks that cross paragraphs, detach code from explanations, include repeated headers/footers, and lose the chapter hierarchy that makes technical documentation navigable.

Current implementation

`app/ingestion/chunker.py` uses PyMuPDF blocks, the embedded table of contents, and printed-heading detection. A block preserves a coherent layout unit (typically a paragraph, heading, or code block). Before indexing, it normalizes common PDF noise, removes recurring marginal text, ignores isolated page numbers, and identifies Vulkan API names and VUIDs.

```
# app/ingestion/chunker.py
for block in page.get_text("blocks", sort=True):
    text = normalize_text(block[4])
    if not text or (is_margin and marginal_fingerprint(text) in ignored_margins):
        continue
    is_code = is_code_block(text)
    blocks.append(TextBlock(page_number, text, is_code, ...))

def extract_api_symbols(text):
    patterns = [r"\bvk[A-Z][A-Za-z0-9_]*\b", r"\bVk[A-Z][A-Za-z0-9_]*\b",
               r"\bVK[A-Z0-9_]+\b", r"\bVUID-[A-Za-z0-9_-]+\b"]
```

The TOC becomes a hierarchical path such as *Drawing a triangle > Presentation > Swap chain*. A deterministic hash of document name, section path, and section start page becomes the section key. This makes section identity stable across a re-ingestion as long as the source structure is stable.

3. Chunking strategy: structure-aware, parent-child, and bounded overlap

This is **structure-aware parent-child chunking**, not pure semantic chunking. The parent is a complete TOC-derived section; children are retrieval-sized chunks made by packing whole layout blocks. The system ranks small children for precision, then recovers nearby or full parent context for generation.

Design choice	Current behavior	Reason
Chunk budget	350 words	Small enough for focused retrieval and cross-encoder scoring; large enough to retain a useful technical explanation.
Overlap	Up to 60 words, using complete trailing blocks	Preserves a boundary concept without slicing sentences or code arbitrarily.
Unit of packing	Paragraph/code layout blocks	Protects code and paragraphs from fixed-token fragmentation.
Parent	Full structural section, stored separately	Lets answer generation see broader explanation without making every vector too broad.

```
# pack_blocks: retain coherent blocks; overlap only the tail blocks
if current and current_words + unit_words > chunk_size:
    packed.append("\n\n".join(item.text for item in current), ...)
    # retain up to overlap_words from complete tail blocks
    current, current_words = overlap, overlap_count
```

Why this over alternatives?

Fixed token windows

Cheap and common baseline, but split code, headings, and lists. Your design improves document fidelity with modest extra parsing complexity.

Semantic breakpoint chunking

Can adapt to topic shifts using embeddings, but is less predictable, can overfit embedding noise, and does not inherently preserve the PDF's chapter hierarchy.

One vector per section

Excellent context but weak precision: a long section averages several ideas into one vector. Parent-child gets precision first, context second.

Late-interaction / CoBERT

Often stronger for exact token interactions, but requires a different index and higher operational/storage complexity. It is a future retrieval experiment, not a free replacement.

Honest limitation: code detection is heuristic and the PDF parser depends on the source layout. For a broad multi-document corpus, add parser regression tests per document family and inspect extraction failures.

4. Persistence and embedding representation

Ingestion stores DocumentSection parents and DocumentChunk children in PostgreSQL. Each child has source page ranges, a section foreign key, API-symbol metadata, text, and a 768-dimensional pgvector embedding. Page metadata is essential: it supports UI citations and page-level evaluation.

```
embedding_text = build_embedding_text(  
    document_name=chunk.document_name, section=chunk.section,  
    page_number=chunk.page_number, content=chunk.content,  
    api_symbols=chunk.api_symbols)  
embeddings = model.encode_document(texts, batch_size=32,  
    normalize_embeddings=True)
```

The model is **BAAI/bge-base-en-v1.5**. BGE is a good quality/latency baseline for semantic retrieval. Normalization makes cosine similarity well behaved and consistent with the pgvector cosine operator. The model object is cached once with `lru_cache`, avoiding repeated downloads/load time.

5. Retrieval: dense, lexical, hybrid, and fusion

Dense retrieval

A query is embedded with `encode_query(..., normalize_embeddings=True)`. PostgreSQL orders chunks by pgvector cosine distance and returns the top candidate set (default 20). Dense retrieval catches semantic paraphrases: “how does presenting work?” can retrieve swap-chain passages even if the wording differs.

```
distance = DocumentChunk.embedding.cosine_distance(query_embedding)
statement = select(DocumentChunk, distance.label("distance")).order_by(distance).limit(limit)
```

BM25 retrieval

BM25 is exact/lexical retrieval. It is particularly valuable in Vulkan because identifiers are meaningful: `VUID-vkQueueSubmit-pWaitSemaphores-03238`, `VK_PIPELINE_STAGE_2_TRANSFER_BIT`, and `vkCmdPipelineBarrier2` should not be treated like ordinary prose.

```
return re.findall(
    r"VUID-[A-Za-z0-9_-]+|VK_[A-Z0-9_]+|vk[A-Za-z0-9_-]+|Vk[A-Za-z0-9_-]+|[A-Za-z0-9]+", text)
```

The current BM25 index is built in process from all stored chunks whenever `BM25Retriever()` is created. That is fine for this small corpus and makes the lexical index simple to reason about; it is not the final design for a large, frequently updated corpus.

Why hybrid search?

Dense search has recall for meaning; BM25 has precision for rare tokens. Neither dominates on all questions. Hybrid retrieval protects the system from dense failures on identifiers and BM25 failures on paraphrases.

Why Reciprocal Rank Fusion (RRF), not raw-score averaging?

Cosine similarity and BM25 scores live on unrelated scales. RRF combines *rank positions*, so no fragile calibration is needed. A result gets $1 / (60 + \text{rank})$ from each list. A passage that both methods like rises; a strong one-method result still survives.

```
combined[chunk_id]["score"] += 1.0 / (rrf_constant + result.rank)
# default rrf_constant = 60
```

Alternative	Why not the current default?
Weighted score normalization	Requires calibrating incomparable scores and retuning when corpus/model changes.
Learned fusion	Potentially stronger, but needs labeled training data and feature monitoring.
Dense only	Weak for VUIDs and exact API constants.
BM25 only	Misses natural-language paraphrases and conceptual questions.

6. Cross-encoder reranking and context reconstruction

After RRF, the system uses **cross-encoder/ms-marco-MiniLM-L6-v2** to score each (query, chunk) pair jointly. Bi-encoder retrieval compares independently precomputed vectors, which is fast but approximate. A cross-encoder reads query and passage together and can model fine-grained relevance; it is more expensive, so it is used only on a small candidate set.

```
pairs = [(query, candidate.chunk.content) for candidate in candidates]
scores = get_reranker().predict(pairs)
ranked = sorted(scored_candidates, key=lambda c: c.reranker_score, reverse=True)
return ranked[:limit] # default 5
```

Important interview honesty: the existing benchmark reports should decide whether reranking is enabled by default, not intuition. Earlier measurements showed reranking can reduce quality on the current evaluation set. That does not mean cross-encoders are “bad”; it means candidate depth, model/domain fit, labels, or chunk construction need calibration. A production-minded engineer reports that result and investigates it.

Parent sliding window

Ranking a single child is precise but sometimes too narrow to answer safely. After ranking, `add_parent_context` fetches chunks with neighboring `chunk_index` values in the same `section_id`. `context_window=1` means the hit plus one sibling on either side. `parent_context=true` also attaches the full parent section.

```
where(DocumentChunk.chunk_index.between(
    chunk.chunk_index - window_size, chunk.chunk_index + window_size))
where(DocumentChunk.section_id == chunk.section_id)
```

This sequence matters: **retrieve narrowly, expand afterward**. Expanding before vector search would dilute child precision; expanding after ranking retains local continuity for the generator.

Current caveat: child `chunk_index` is global ingestion order, then constrained by section ID. It works because section filtering is applied. A more explicit `position_in_section` column would make the model clearer and safer for multi-document ingestion.

7. Grounded answer generation and citations

The LLM is optional. The retrieval endpoint works without it; the answer endpoint requires an OpenAI-compatible model endpoint, such as local Ollama Qwen. The system passes only numbered retrieved passages, asks for a short answer, and requires `[n]` citations for factual claims.

```
prompt = "Answer only from the supplied Vulkan documentation passages. " +
    "Use at most three concise sentences. Cite every factual claim using [n]."
payload = { "temperature": 0, "response_format": {"type": "json_object"},
    "reasoning_effort": "none" }
```

Output handling removes `<think>` traces, parses JSON, rejects empty/ellipsis placeholders, retries once, then filters returned metadata down to citations that the answer actually referenced. An asyncio lock serializes generation, a practical protection for a small local model that can become unstable or slow under concurrent requests.

8. API, operations, and evaluation

API contract

FastAPI exposes `POST /v1/search` for evidence and `POST /v1/answers` for cited answers; `GET /health` verifies Postgres connectivity and `GET /metrics` exposes basic request/error/latency counters. Pydantic limits query length and result count. Startup creates the pgvector extension/table/index and warms the embedding and reranker models.

The database creates an HNSW cosine index. HNSW is an approximate-nearest-neighbor choice that improves vector retrieval as the corpus grows, unlike a linear scan. PostgreSQL also keeps source metadata and relational parent/child links in one operational system.

Evaluation methodology

`data/evaluation/vulkan_retrieval.json` contains page-level relevance labels. The evaluator records Recall@k, MRR@k, nDCG@k and P50/P95 latency for reranked and non-reranked configurations. It deliberately deduplicates relevant pages before nDCG, preventing overlapping chunks from inflating the metric.

```
new_pages = (result_pages & relevant_pages) - covered_pages
if new_pages:
    relevant_ranks.append(rank)
    covered_pages.update(new_pages)
```

Metric	Interview explanation
Recall@k	Did the top-k cover the known source pages? Good for evidence availability.
MRR@k	How early is the first useful result? Good for answer quality and user experience.
nDCG@k	Does the ranked list place distinct relevant evidence near the top?
P50 / P95 latency	Median experience and tail risk; reranking/generation can affect P95 substantially.

Evaluation rule: change one retrieval variable at a time, run the same labeled set, compare quality and latency, inspect failed queries, then decide whether the extra complexity is justified.

9. What is production-ready, and what is next?

This is a credible production-minded foundation: grounded sources, API validation, PostgreSQL/pgvector, HNSW, CORS, health/metrics, model warmup, citations, error handling, an evaluation harness, CI tests, and a user-facing evidence viewer.

Next improvement	Why it matters
Persistent BM25 / search engine	Current BM25 is rebuilt in memory; move it to Elasticsearch/OpenSearch or a persisted lexical index for scale and updates.
Alembic migrations	Replace lightweight startup ALTER statements with versioned, reviewable schema migrations.

Observability	Add structured logs, traces, per-stage latency, retrieval IDs, model version, and citation/answer quality monitoring.
Evaluation expansion	More adversarial VUIDs, negative queries, section/chunk labels, human faithfulness review, and regression gates.
Reranker tuning	Test candidate depth, domain rerankers, and score/quality deltas before making it a hard default.
Security + scale	Auth, rate limits, connection pools, async/background ingestion, document ACLs, and tenant isolation if multi-user.

10. Interview answers to rehearse

“Why not just send the whole PDF to the model?”

Context is limited, expensive, and less reliable when irrelevant text crowds out evidence. Retrieval selects a small, attributable context; parent expansion restores only the necessary neighborhood.

“Why do you need BM25 if you have embeddings?”

Technical documentation contains symbolic identifiers and error IDs. BM25 preserves exact-match signal, while embeddings handle paraphrase. Hybrid fusion improves robustness across both query types.

“Why parent-child retrieval?”

Small children create discriminative vectors and accurate ranking. Full sections preserve explanation. I rank children, then attach sibling/full parent context after ranking instead of sacrificing precision up front.

“How do you prevent hallucinations?”

I constrain the prompt to retrieved passages, require numbered citations, return only metadata for citations actually mentioned, and make “not supported by passages” an allowed response. This reduces, but does not mathematically eliminate, hallucination—so I also evaluate retrieval and inspect answers.

“What would you improve first?”

I would expand and harden evaluation, because it tells me which change matters. In parallel I would persist lexical search and add proper migrations/observability before claiming large-scale production readiness.

Final ownership checklist

- I can trace one query from API payload to embedding, BM25, RRF, reranker, context expansion, prompt, answer, and citation.
- I can state the exact reason hybrid retrieval exists: semantic recall plus symbolic precision.
- I can distinguish retrieval evaluation from generation evaluation.
- I can name current limitations without defensiveness and propose a measurable next experiment.

This guide describes the codebase as implemented at the time of generation. Re-run the benchmark after changing chunking, models, candidate limits, or the source corpus.